

Lecture 9: Data Visualization – Base R

Harris Coding Camp – Standard Track

Summer 2024

Data Visualization: Motivation

Suppose we want to know the following:

- ▶ What is the annual housing sales in Texas over time?
- ▶ Can we learn about the relevant housing prices of one city to another in Texas?

We probably want to know the *average* of housing price, how much the price *varies* over time, and info about the *extremes*

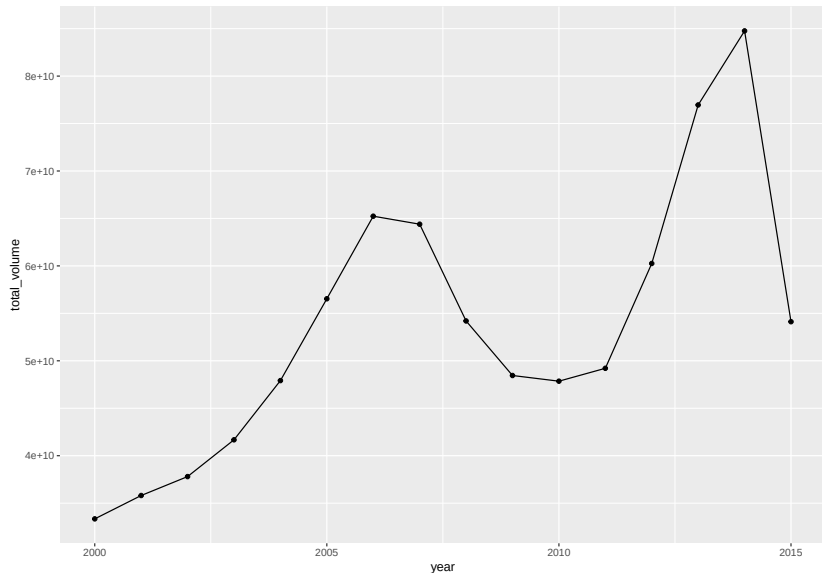
Data Visualization: Motivation

Is it helpful to simply look at all these data?

	city	year	month	sales	volume	median	listings	inventory	date
171	Abilene	2014	3	165	21615979	112400	929	5.1	2014.167
172	Abilene	2014	4	178	25070931	124500	965	5.4	2014.250
173	Abilene	2014	5	191	26615130	129300	1001	5.7	2014.333
174	Abilene	2014	6	230	34398506	135200	1023	5.8	2014.417
175	Abilene	2014	7	231	35861350	145800	1033	5.8	2014.500
176	Abilene	2014	8	203	29671329	129200	1024	5.8	2014.583
177	Abilene	2014	9	201	30904319	135900	1012	5.8	2014.667
178	Abilene	2014	10	190	25431388	122400	987	5.6	2014.750
179	Abilene	2014	11	158	24120554	132200	941	5.3	2014.833
180	Abilene	2014	12	165	23959468	130000	823	4.6	2014.917
181	Abilene	2015	1	158	23486998	134100	801	4.4	2015.000
182	Abilene	2015	2	151	19834263	126500	767	4.1	2015.083
183	Abilene	2015	3	198	31869437	136800	821	4.4	2015.167
184	Abilene	2015	4	201	28301159	129600	891	4.7	2015.250
185	Abilene	2015	5	199	31385757	144700	919	4.8	2015.333
186	Abilene	2015	6	260	41396230	141500	965	5.0	2015.417
187	Abilene	2015	7	268	45845730	148700	986	5.0	2015.500
188	Amarillo	2000	1	102	8860000	80000	972	5.3	2000.000
189	Amarillo	2000	2	147	13875000	78300	937	5.1	2000.083
190	Amarillo	2000	3	201	17930000	74800	995	5.5	2000.167
191	Amarillo	2000	4	176	16955000	87700	985	5.5	2000.250
192	Amarillo	2000	5	198	19420000	81100	1020	5.6	2000.333
193	Amarillo	2000	6	206	20035000	84100	1055	5.9	2000.417
194	Amarillo	2000	7	190	20430000	94300	1193	6.8	2000.500
195	Amarillo	2000	8	242	25995000	95300	1208	6.6	2000.583
196	Amarillo	2000	9	158	16000000	91000	1248	6.9	2000.667
197	Amarillo	2000	10	165	16810000	89000	1191	6.7	2000.750
198	Amarillo	2000	11	133	13955000	87300	1174	6.7	2000.833
199	Amarillo	2000	12	158	15800000	88300	1092	6.3	2000.917
200	Amarillo	2001	1	134	14445000	89000	1087	6.2	2001.000
201	Amarillo	2001	2	157	15345000	87100	1037	5.9	2001.083
202	Amarillo	2001	3	187	19260000	86300	1030	5.9	2001.167
203	Amarillo	2001	4	199	21740000	93200	1055	6.0	2001.250
204	Amarillo	2001	5	225	22225000	88300	1064	5.9	2001.333

Data Visualization: Motivation

What if we make a plot of annual housing sales over time. . .



Data Visualization: Introduction

- ▶ Be able to generate plots (i.e. visualize data)
- ▶ Know what types of figures you want
- ▶ Use plots to gain information
 - ▶ graphs, including axes, must be labeled properly to allow readers to understand the visualization with ease
 - ▶ variable scales should be portrayed accurately
 - ▶ graphs should be as intuitive as possible

Data Visualization: Introduction

Two main frameworks which create graphics in R:

The **Base R** framework, and the **tidyverse/ggplot** framework

- ▶ Base R consists of about 30 packages that are always loaded automatically when we open R
- ▶ Base R can and is still commonly used to create visualizations, though more visualizations are now generated using the **tidyverse/ggplot** framework

Getting started

We will use a built-in dataset called `airquality`: daily air quality measurements in New York, May to September 1973 stored within a data frame with 153 observations. The data set contains:

- ▶ `Ozone`: Mean ozone in parts per billion (ppb)
- ▶ `Solar.R`: Solar radiation in Langleys (lang)
- ▶ `Wind`: Average wind speed in miles per hour (mph)
- ▶ `Temp`: Maximum daily temperature in degrees Fahrenheit
- ▶ `Month`: Month (1–12)
- ▶ `Day`: Day (1–31)

Getting started

Let's look at the data:

```
head(airquality)
```

##		Ozone	Solar.R	Wind	Temp	Month	Day
##	1	41	190	7.4	67	5	1
##	2	36	118	8.0	72	5	2
##	3	12	149	12.6	74	5	3
##	4	18	313	11.5	62	5	4
##	5	NA	NA	14.3	56	5	5
##	6	28	NA	14.9	66	5	6

Simplest plot code structure

```
plot(y ~ x,                                # plot y by x
     type = "[type]",                      # type (points, lines, or both)
     data = [data_name],                   # data source
     ylab = "[y variable]",               # add y-axis label
     xlab = "[x variable]",               # add x-axis label
     main = "[title]"                     # add title
)
```

Understanding plot(): Scatter plot

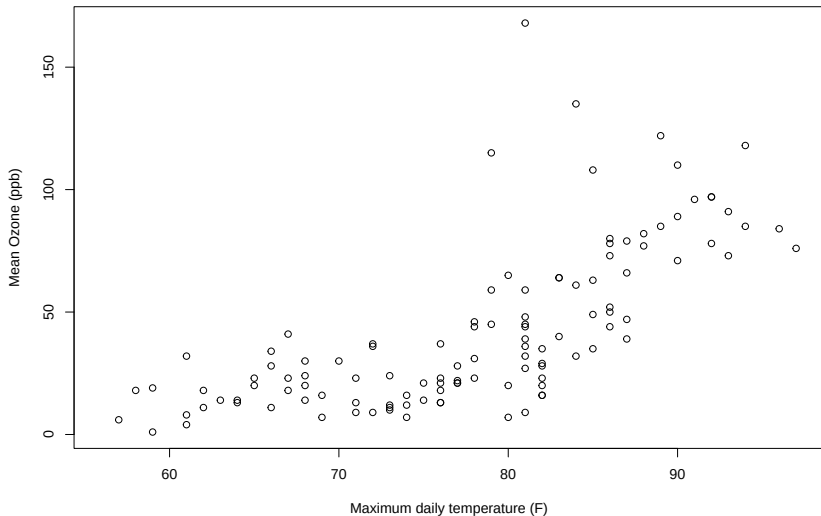
Let's get started!

The simplest graph is a **scatter plot**. Scatter plots are used when the graph is set up to display the relationship between two variables.

```
plot(Ozone ~ Temp,           # plot Ozone by Temperature
     type = "p",            # plot type "p" (points)
     data = airquality,      # data from airquality
     ylab = "Mean Ozone (ppb)", # add y-axis label
     xlab = "Maximum daily temperature (F)", # add x-axis label
     main = "Temperature vs ozone in New York City" # add title
)
```

Understanding plot(): Scatter plot

Temperature vs ozone in New York City



Scatter plot + Vertical lines

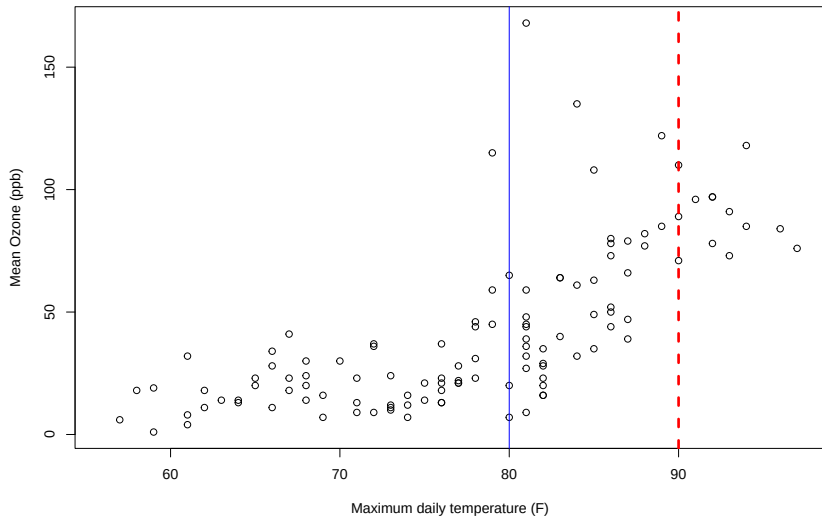
Often, we may include more info by adding some **vertical lines** to a plot:

Syntax: `abline(v = [number], <other options>)`

```
plot(Ozone ~ Temp,           # plot Ozone by Temperature
     type = "p",             # plot type "p" (points)
     data = airquality,      # data from airquality
     ylab = "Mean Ozone (ppb)", # add y-axis label
     xlab = "Maximum daily temperature (F)", # add x-axis label
     main = "Temperature vs ozone in New York City" # add title
)
abline(                       # add a line
  v = c(80, 90),             # draw two "v" (vertical) lines
  col=c("blue", "red"),      # define line colors
  lty=c(1,2),                 # define line types
  lwd=c(1,3)                  # define line widths
)
```

Scatter plot + Vertical lines

Temperature vs ozone in New York City



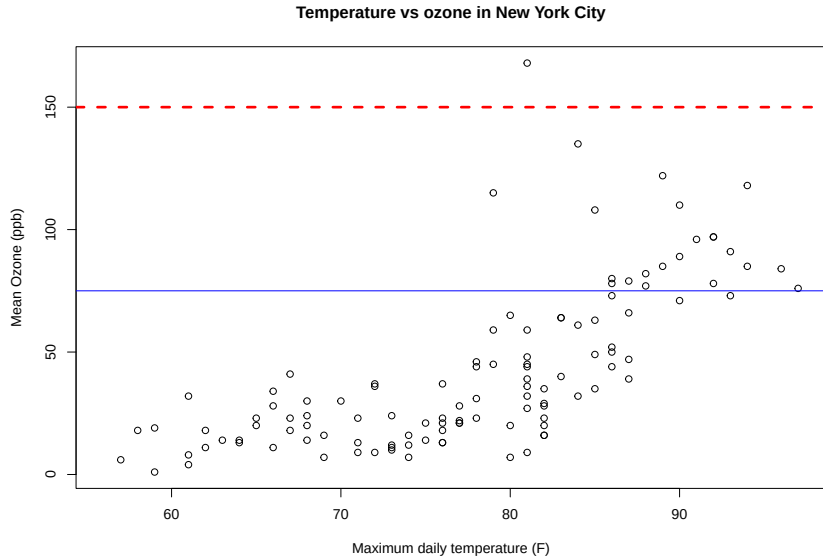
Scatter plot + Horizontal lines

...or by adding some **horizontal lines** to a plot:

Syntax: abline(h = [number], <other options>)

```
plot(Ozone ~ Temp,           # plot Ozone by Temperature
     type = "p",             # plot type "p" (points)
     data = airquality,      # data from airquality
     ylab = "Mean Ozone (ppb)", # add y-axis label
     xlab = "Maximum daily temperature (F)", # add x-axis label
     main = "Temperature vs ozone in New York City" # add title
)
abline(                       # add a line
  h = c(75, 150),            # draw two "h" (horizontal) lines
  col=c("blue", "red"),      # define line colors
  lty=c(1,2),                 # define line types
  lwd=c(1,3)                  # define line widths
)
```

Scatter plot + Horizontal lines



Scatter plot + tendency line

We may add a **linear (regression) line** to the scatter plot to see the trend, and change the color of the points:

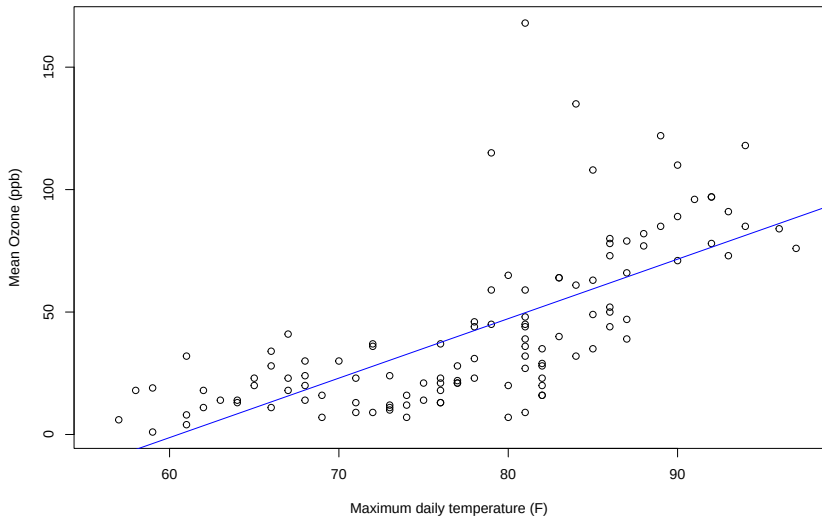
Syntax: `abline(lm(df_name$y ~ df_name$x), <other options>)`

```
reg <- lm(Ozone ~ Temp, data = airquality)
```

```
plot(Ozone ~ Temp,           # plot Ozone by Temperature
     type = "p",            # plot type "p" (points)
     data = airquality,      # data from airquality
     ylab = "Mean Ozone (ppb)", # add y-axis label
     xlab = "Maximum daily temperature (F)", # add x-axis label
     main = "Temperature vs ozone in New York City" # add title
)
abline(reg,                  # add a line of linear model
       col="blue"           # define line color as blue
)
```

Scatter plot + tendency line

Temperature vs ozone in New York City

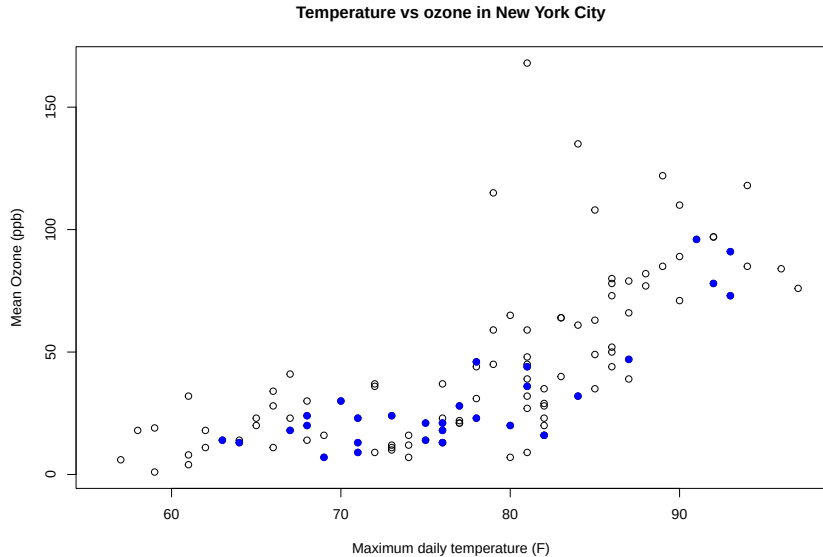


Scatter plot with subsetting

Here, we start with the same plot as above, and then annotate it by coloring blue the data points corresponding to the month of September:

```
plot(Ozone ~ Temp,           # plot Ozone by Temperature
     type = "p",            # plot type "p" (points)
     data = airquality,     # data from airquality
     ylab = "Mean Ozone (ppb)",      # add y-axis label
     xlab = "Maximum daily temperature (F)", # add x-axis label
     main = "Temperature vs ozone in New York City" # add title
)
points(airquality$Temp[airquality$Month == 9],
       airquality$Ozone[airquality$Month == 9],
       pch = 19,          # solid circle
       col = "blue")
```

Scatter plot with subsetting



Scatter plot with subsetting + legend

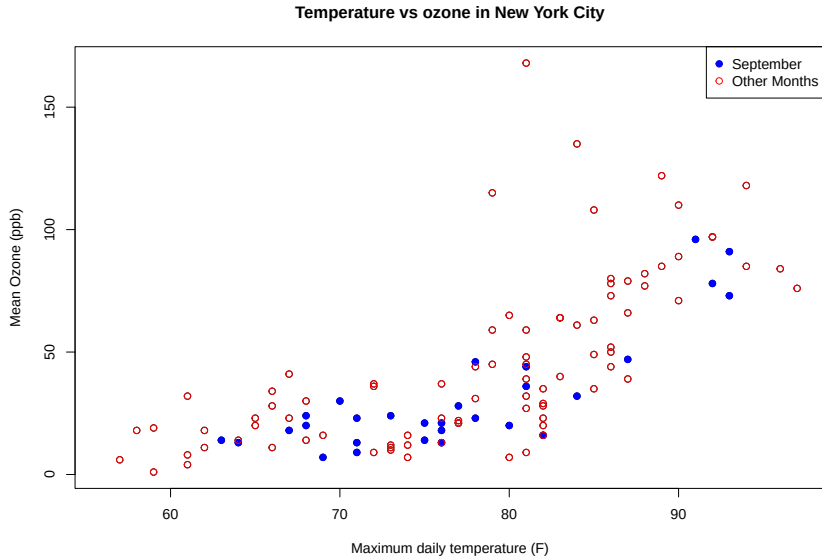
Say we'd like to generate a plot which colors the data points for the month of September **blue** and colors all of the other data points **red**.

Also, we add a **legend** explaining the meaning of the different colors in the plot.

```
plot(Ozone ~ Temp,           # plot Ozone by Temperature
     type = "p",            # plot type "p" (points)
     data = airquality,     # data from airquality
     ylab = "Mean Ozone (ppb)", # add y-axis label
     xlab = "Maximum daily temperature (F)", # add x-axis label
     main = "Temperature vs ozone in New York City" # add title
)

points(airquality$Temp[airquality$Month == 9],
       airquality$Ozone[airquality$Month == 9],
       pch = 19, col = "blue")
points(airquality$Temp[airquality$Month != 9],
       airquality$Ozone[airquality$Month != 9],
       col = "red")
legend("topright",
      pch = c(19, 1),
      col = c("blue", "red"),
      legend = c("September", "Other Months"))
```

Scatter plot with subsetting + legend



Try it yourself!

1. Use the code below to load the data set and call it dta9_1.

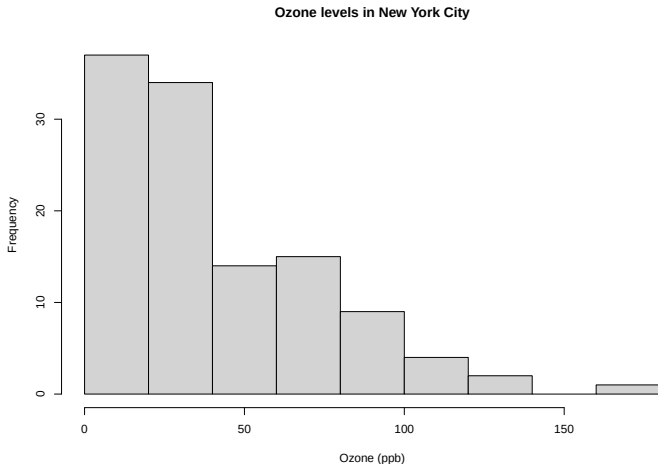
```
dta9_1 <-  
base::readRDS(url("https://slcladal.github.io/data/d03.rda","rb"))
```

2. Take a look at dta9_1. How many variables and observations are included?
3. Find the mean and standard deviation of Variable1 and Variable2.
4. Then, create a **scatter plot** showing Variable1 on the x-axis and Variable2 on the y-axis. You may change the color of the points.
5. Finally, add a linear tendency line. By eyeballing the plot, can you determine the linear relationship between the two variables?

Histogram

It summarizes numeric variables by showing their distribution across bins.

```
hist(airquality$Ozone,  
     xlab = "Ozone (ppb)",  
     main = "Ozone levels in New York City")
```

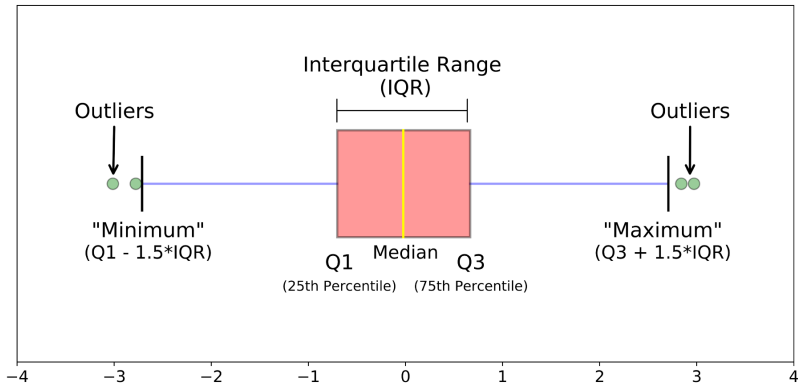


Box plot

Boxplots show the relationships between categorical and numeric variables.

They are very useful because they not only provide measures of central tendency (e.g. median) but also offer info about the distribution (e.g. 1st and 3rd quartile, min, max) of the data.

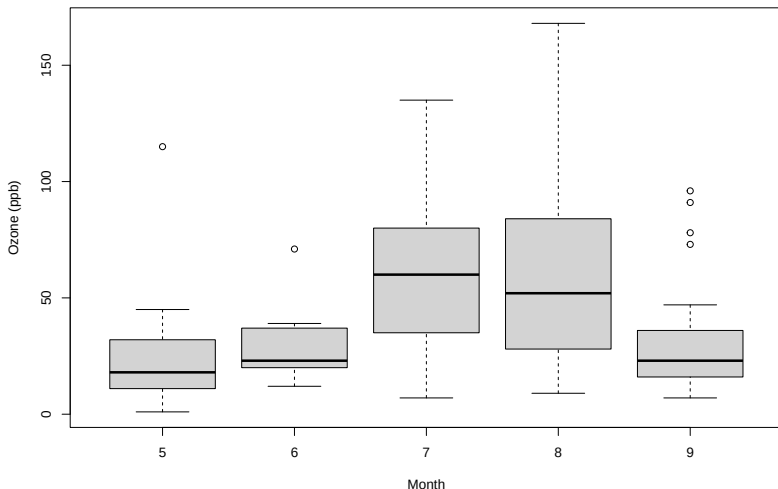
Any data points beyond 1.5 times the IQR of the data are indicated separately with circles.



Box plot

Here, we are plotting ozone levels in New York **by month**.

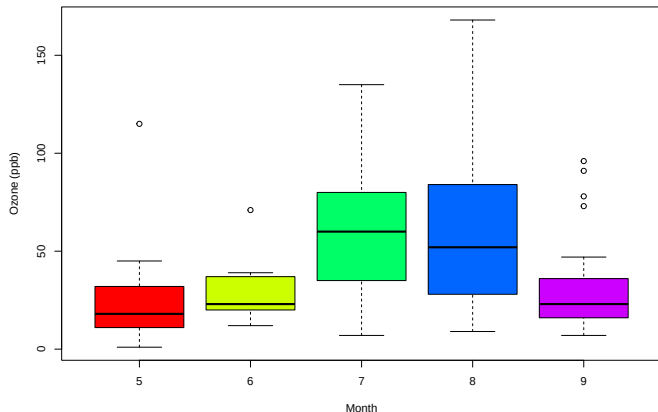
```
boxplot(Ozone ~ Month, data = airquality,  
        xlab = "Month", ylab = "Ozone (ppb)")
```



Box plot

We could color the boxes individually by specifying a vector of number with the `rainbow()` functions:

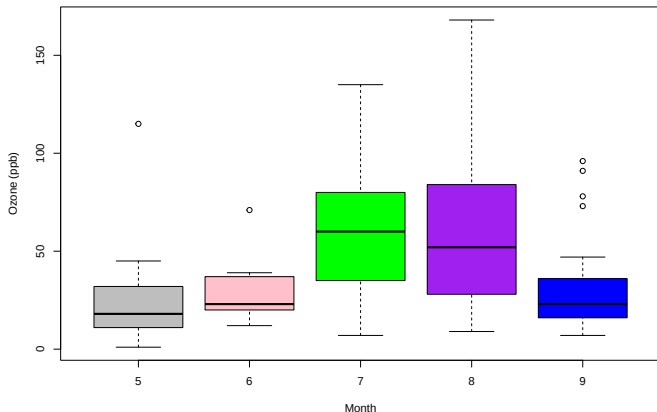
```
boxplot(Ozone ~ Month, data = airquality,  
        col = rainbow(5),  
        xlab = "Month", ylab = "Ozone (ppb)")
```



Box plot

We could also color the boxes individually by specifying a vector of number with the `c()` functions:

```
boxplot(Ozone ~ Month, data = airquality,  
        col = c("grey", "pink", "green", "purple", "blue"),  
        xlab = "Month", ylab = "Ozone (ppb)")
```



Try it yourself: Plot 1

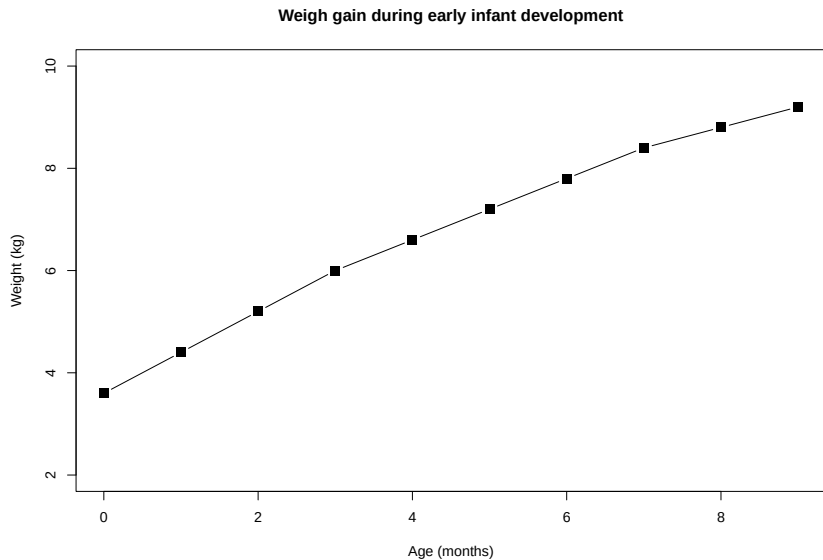
1. First, load the dataset `weight_chart.txt` and give it a name (e.g. `weight_chart`). Take a look at the dataset.

```
# read the space separated data frame using `read.delim()`  
weight_chart <- read.delim("weight_chart.txt")  
weight_chart
```

2. Then, try to replicate the plot in the next slide by modifying the following code. Hint: When choosing type, use "b" (which means *both* line and points).

```
plot(??? ~ ???,                                # fill in appropriate X and Y  
     type = "b",                               # plot type "b" (both line and points)  
     pch = 15,                                 # filled square  
     cex = 1.5,                               # size of pch symbols  
     data = weight_chart,                     # data source  
     ylim = c(2,10),                          # y-axis limits  
     ylab = "???",  
     xlab = "???",  
     main = "???"  
)
```

Try it yourself: Plot 1

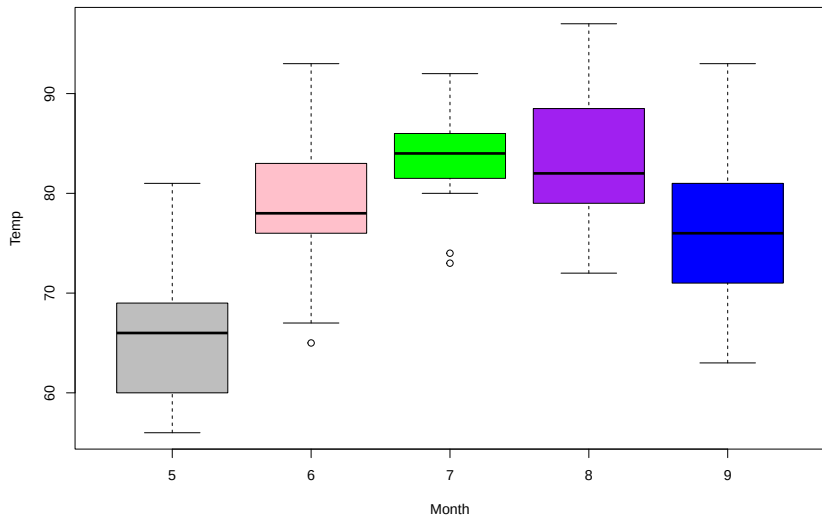


Try it yourself: Plot 2

Use the dataset `airquality` and try to replicate the plot in the next slide by modifying the following code.

```
"which_plot"(??? ~ ???,  
              data = airquality,  
              col = c("???", "pink", "green", "???", "???"))
```

Try it yourself: Plot 2



Recap

- ▶ Visualizing our data can help lead to powerful insights between variable relationships
 - ▶ Making quick plots helps us understand data and makes us aware of data issues
- ▶ We've learned how to make commonly used plots, including:
 - ▶ scatter plots (with auxiliary lines)
 - ▶ histograms
 - ▶ box plots
- ▶ There are many ways you can visualize your data
 - ▶ We can also use `ggplot()` to generate plots (Lecture 10)

Appendix: More plots

Multiple scatter plots

Making multiple plots **side by side** is a useful way to visualize many relationships between variables.

In the example below, we make two plots: one of Ozone and Temp and another with Ozone and Wind.

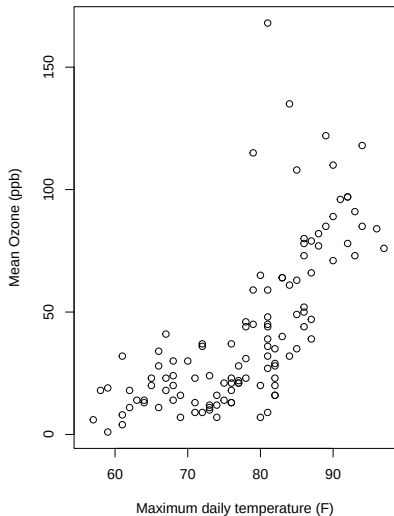
```
par(mfrow = c(1, 2))          # we create one row of plots and two columns

plot(Ozone ~ Temp,             # plot Ozone by Temperature
     type = "p",               # plot type "p" (points)
     data = airquality,        # data from airquality
     ylab = "Mean Ozone (ppb)", # add y-axis label
     xlab = "Maximum daily temperature (F)", # add x-axis label
     main = "Temperature vs ozone in New York City" # add title
)

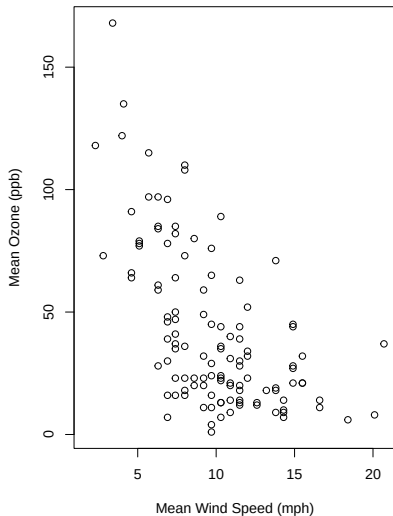
plot(Ozone ~ Wind,             # plot Ozone by Wind speed
     type = "p",               # plot type "p" (points)
     data = airquality,        # data from airquality
     ylab = "Mean Ozone (ppb)", # add y-axis label
     xlab = "Mean Wind Speed (mph)", # add x-axis label
     main = "Wind Speed vs ozone in New York City" # add title
)
```

Multiple scatter plots

Temperature vs ozone in New York City



Wind Speed vs ozone in New York City



Line Plot

It is particularly useful in time series or finance.

```
plot(airquality$Temp,  
     type = "l",  
     ylab = "Temperature", main = "")
```

